

STAYVEO

PREMIUM STUDENT LIVING SUITE

System & Security Architecture

Production Architecture, Security Model,
Scalability Strategy & Migration Plan

VERSION	v1.0.0
DATE	2026-09-08
STATUS	Active Development
MATURITY	Pre-Production
CLASSIFICATION	Confidential — Internal
PREPARED FOR	StayVeo Engineering

Table of Contents

Page numbers are populated after final rendering.

1. Executive Summary
2. Architecture Overview
3. Users & Actors
4. DNS Architecture
5. CDN / Edge Architecture
6. WAF & DDoS Protection
7. Load Balancing
8. Backend Infrastructure
9. Authentication Architecture
10. OTP Architecture
11. Session / JWT / Cookie Architecture
12. Authorization Architecture
13. RBAC Model
14. Ownership Model
15. API Security
16. Redis / Caching
17. Database Architecture
18. Database Security / RLS
19. Storage Security
20. Secrets Management
21. Logging & Monitoring
22. Backup & Disaster Recovery
23. Threat Model
24. Security Testing
25. Scaling Strategy
26. Cost Strategy
27. Current Architecture
28. Target Architecture
29. Migration Plan
30. Architecture Decision Records
31. Failure Scenarios
32. Security Gap Register
33. Maturity Assessment
34. Request Lifecycle
35. Security Principles
36. Glossary
37. Final Recommendations

Executive Summary

StayVeo is a student-living marketplace connecting students with PG (Paying Guest) accommodation, tiffin (meal subscription) services, and roommate matching. The platform serves two primary user classes — Students and Providers — each with dedicated onboarding flows, dashboards, and feature sets.

This document captures the current technical architecture as implemented in the repository, identifies security gaps, and provides a phased roadmap toward production readiness. Every component is classified by its implementation status: CURRENT, PARTIAL, PLANNED, RECOMMENDED, or NOT IMPLEMENTED.

Technology Stack (Current)

LAYER	TECHNOLOGY	STATUS	NOTES
Frontend	React 19 + Vite 8	CURRENT	SPA with react-router-dom v7
Styling	Vanilla CSS	CURRENT	No framework (Tailwind, etc.)
Backend	Fastify 5 + TypeScript	CURRENT	Node.js 20, tsx for dev
ORM	Prisma 6	CURRENT	Typed schema, migrations
Database	PostgreSQL (Supabase)	CURRENT	Supabase-hosted Postgres
Auth (Frontend)	Supabase Auth SDK	PARTIAL	Session used for display name only
Auth (Backend)	Header-based (x-user-id)	CURRENT	No JWT/session verification
Storage	Supabase Storage	CURRENT	pg-images, KYC buckets
Validation	Zod	CURRENT	Schema validation on some routes
Payment	Mock Provider	CURRENT	Razorpay adapter stubbed
Maps	Mapbox GL	CURRENT	PG/tiffin location display
Notifications	In-app only	PARTIAL	In-memory queue, no push

[CRITICAL]

CRITICAL FINDING: The backend has NO authentication middleware. All API routes trust the x-user-id and x-provider-phone headers sent by the frontend without cryptographic verification. Any client can impersonate any user by sending arbitrary headers. This is the #1 security gap requiring immediate remediation before production deployment.

SECTION 1 ARCHITECTURE OVERVIEW

StayVeo follows a classic Single-Page Application (SPA) architecture with a separate API backend. The frontend is a React application served by Vite during development (and as static files in production). The backend is a Fastify API server that communicates with a Supabase-hosted PostgreSQL database via Prisma ORM.

System Purpose

StayVeo is a marketplace for student accommodation and food services in Indian cities. It enables:

- Students to discover, compare, and book PG accommodations
- Students to subscribe to tiffin (meal) services from verified providers
- Students to find compatible roommates via a matching/swiping system
- Providers to onboard their PG or tiffin businesses and manage day-to-day operations
- Providers to track deliveries, customers, payments, and menu planning

High-Level Architecture Diagram

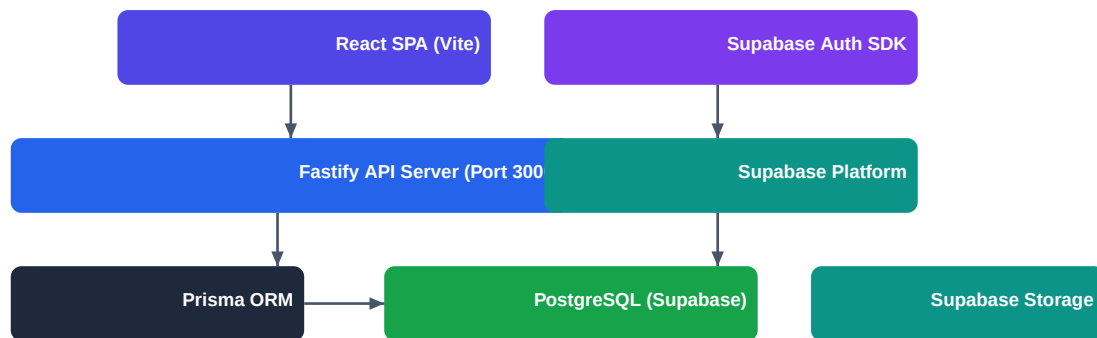


Figure 1.1 — Current StayVeo Architecture (Simplified)

Layer Classification

LAYER	DESCRIPTION	STATUS
Client Layer	React SPA served by Vite dev server / static files	CURRENT
Edge Layer	CDN, WAF, DDoS protection	NOT IMPLEMENTED
Network Layer	DNS, TLS, load balancing	NOT IMPLEMENTED
Application Layer	Fastify API with controllers/services/repos	CURRENT
Authentication Layer	Header-based identity (no verification)	PARTIAL — INSECURE
Authorization Layer	Provider ownership checks in service code	PARTIAL
Data Layer	PostgreSQL via Prisma + Supabase direct queries	CURRENT
Storage Layer	Supabase Storage (pg-images, KYC documents)	CURRENT
Caching Layer	No cache layer exists	NOT IMPLEMENTED
Observability Layer	Pino logger only (stdout)	PARTIAL
Security Layer	CORS, Zod validation, error handler	PARTIAL

SECTION 2 USERS & ACTORS

The system currently supports two user roles defined in the UserRole enum: STUDENT and PROVIDER. There is no ADMIN role in the database schema.

ACTOR	RESPONSIBILITIES	TRUST LEVEL	SECURITY RISKS
Student	Browse PGs, reserve tiffins, manage profile, save listings	Low (untrusted client)	IDOR on user resources, header spoofing
Provider (PG)	Onboard PG, manage rooms, handle bookings, view earnings	Low (untrusted client)	Cross-provider data access, listing manipulation
Provider (Tiffin)	Onboard kitchen, manage menu/deliveries/customers	Low (untrusted client)	Kitchen ID manipulation, menu tampering
Backend API	Business logic, data validation, database access	High (server-side)	Service-role key exposure, injection
Supabase	PostgreSQL hosting, auth, storage, real-time	High (managed service)	Credential leak, RLS bypass
SMS/OTP Provider	OTP delivery for phone verification	Medium (external)	OTP brute force, SMS interception

[WARNING]

There is no Admin role or admin panel in the current codebase. All administrative operations (e.g., kitchen verification) must be performed directly in the database. An ADMIN role is RECOMMENDED for production.

Identity Chain (Current)

The current identity resolution differs between Student and Provider flows:

Student Flow

- Frontend calls /api/v1/auth/send-otp with phone number
- Backend creates a User row (role=STUDENT) if not exists — OTP is DUMMY (accepts any value)
- Frontend stores userId in localStorage and sends it as x-user-id header
- Backend trusts x-user-id header without verification

Provider (Tiffin) Flow

- Provider onboarding uses x-provider-phone header to identify the provider
- Backend resolves ProviderProfile from phone, verifies otpVerified flag
- Kitchen is resolved via ownerId FK from ProviderProfile.id
- Ownership check: only the kitchen's owner profile can modify it

[CRITICAL]

Both flows trust client-supplied headers. A malicious client can send any phone number or user ID to access arbitrary accounts.

SECTION 3

DNS ARCHITECTURE

[NOTE]

STATUS: NOT IMPLEMENTED. The application currently runs on localhost (dev) and Microsoft Dev Tunnels / Cloudflare Quick Tunnels for mobile testing. No production DNS is configured.

The current frontend .env shows API endpoints on Dev Tunnels (zjdd8mbz-3000.inc1.devtunnels.ms). Supabase provides its own domain (zjlwzophfxnbpezgdddp.supabase.co) for database and storage.

Recommended Production DNS

- stayveo.com — Frontend (React SPA static files)
- api.stayveo.com — Backend API (Fastify)
- admin.stayveo.com — Admin panel (when built)
- All domains must use HTTPS with valid TLS certificates
- DNS should be behind a CDN (Cloudflare) to hide origin IP addresses

Why DNS Matters

DNS is the entry point for every request. Exposing the origin server's IP address directly allows attackers to bypass CDN/WAF protections. Using a CDN like Cloudflare as a reverse proxy hides the origin and provides DDoS protection at the DNS level.

SECTION 4

CDN / EDGE ARCHITECTURE

[NOTE]

STATUS: NOT IMPLEMENTED. Static assets are served directly from the Vite dev server or the origin. No CDN is configured.

What Should Be Cached

- Static JS/CSS bundles (immutable, cache indefinitely with content hashes)
- Public images (PG photos, kitchen logos) — long cache TTL
- Fonts, icons, public assets

What Should NOT Be Cached

- API responses containing user-specific data
- Authentication endpoints
- Provider dashboard data
- Payment/reservation endpoints

Recommended Architecture

Use Cloudflare Free plan or Vercel/Netlify edge for the React SPA. API server should be proxied through a CDN with cache disabled for `/api/*` routes.

SECTION 5

WAF & DDOS PROTECTION

[NOTE]

STATUS: NOT IMPLEMENTED. No WAF, rate limiting, or DDoS protection is currently in place.

A Web Application Firewall (WAF) inspects incoming HTTP requests and blocks malicious payloads before they reach the application server.

Why WAF Should Be Before Application Servers

Placing the WAF before the backend means attacks are blocked at the edge, reducing load on application servers and preventing exploit payloads from ever reaching business logic.

Recommended Timeline

WAF is NOT needed immediately during development. It should be introduced when the application receives real user traffic (Stage 2 scaling). Cloudflare Free provides basic WAF rules and DDoS protection at no cost.

SECTION 6 LOAD BALANCING

[NOTE]

STATUS: NOT IMPLEMENTED. The application runs as a single Fastify process. There is no load balancer, no horizontal scaling.

A load balancer distributes incoming API requests across multiple backend server instances, providing redundancy and increased throughput.

Component Clarification

COMPONENT	PURPOSE	EXAMPLE
CDN	Cache static assets at edge locations globally	Cloudflare, CloudFront
WAF	Filter malicious HTTP requests	Cloudflare WAF, AWS WAF
Reverse Proxy	Terminate TLS, route to backend	Nginx, Caddy
Load Balancer	Distribute traffic across multiple servers	Nginx, HAProxy, ALB
API Gateway	Rate limit, auth, routing, throttle	Kong, AWS API Gateway

Load balancing is NOT needed until StayVeo has enough traffic to exceed a single server's capacity. For early production, a single server with PM2 cluster mode is sufficient.

SECTION 7 BACKEND INFRASTRUCTURE

The backend is a Fastify 5 API server written in TypeScript, using tsx for development hot-reloading. It follows a layered architecture pattern.

Architecture Pattern

Routes !' Controller !' Service !' Repository !' Prisma !' PostgreSQL

- Routes (tiffin.routes.ts, auth.routes.ts, etc.): Register HTTP endpoints with Fastify
- Controllers: Extract request data, call services, format responses
- Services: Business logic, validation, orchestration
- Repositories: Database access via Prisma (used in some modules)
- Prisma: ORM generating typed queries from schema.prisma

Registered API Modules

MODULE	PREFIX	FILES
Auth	/api/v1/auth	auth.routes/controller/service/repo
Users	/api/v1/users	user.routes/controller/service
Student	/api/v1/student	student.routes/controller/service
Provider	/api/v1/provider	provider.routes/controller/service
PG	/api/v1/pg	pg.routes/controller/service
Tiffin	/api/v1/tiffin	tiffin.routes + 6 service files
Tiffin Provider	/api/v1/tiffin/provider/*	tiffin-provider.controller/service
Bookings	/api/v1/bookings	booking.routes/controller/service
Payments	/api/v1/payments	payment.routes/controller/service
Notifications	/api/v1/notifications	17 files (full pipeline)
Room Listings	/api/provider/room-listings	room-listing.routes
Media	/api/v1/media	media.routes
Saved	/api/v1/saved	saved.routes
Colleges	/api/v1/colleges	college.routes

[WARNING]

No authentication middleware exists. Every route is publicly accessible to any client that provides the expected headers. The global error handler catches Prisma and Zod errors but does not enforce auth.

SECTION 8

AUTHENTICATION ARCHITECTURE

[CRITICAL]

CRITICAL: The backend currently has NO authentication middleware. There is no JWT verification, no session validation, no Supabase Auth token checking on the server side. All routes are open.

Current Authentication State

The application has TWO separate authentication paths that do NOT share a verified identity:

Path 1: Student Auth (Dummy OTP)

- POST /api/v1/auth/send-otp — accepts any phone, creates User if not exists
- POST /api/v1/auth/verify-otp — accepts ANY OTP value (dummy mode)
- No JWT/session is created — userId stored in localStorage
- Subsequent requests send x-user-id header — backend trusts it blindly

Path 2: Provider Auth (Supabase Auth)

- Frontend uses Supabase Auth SDK for phone OTP (real Supabase OTP)
- Provider profile created via /api/provider/verify-otp
- Subsequent requests send x-provider-phone header — backend trusts it
- The resolveOwner() function verifies otpVerified flag on ProviderProfile

[NOTE]

Authentication = "Who are you?" — The current system accepts identity claims without cryptographic proof.

Recommended Architecture

Use Supabase Auth as the single source of identity for ALL users. The backend should verify the Supabase JWT access token on every request.

1. Frontend authenticates via Supabase Auth (phone OTP)
2. Supabase issues JWT access token + refresh token
3. Frontend includes JWT in Authorization: Bearer <token> header
4. Backend middleware verifies JWT signature using Supabase JWT secret
5. Verified auth.uid() maps to public.users via user_id
6. Backend passes verified user to route handlers

SECTION 9

OTP ARCHITECTURE

Current OTP State

[CRITICAL]

Student OTP is DUMMY — the auth.service.ts comment reads: "OTP is DUMMY — any value is accepted". No rate limiting, no CAPTCHA, no attempt tracking exists for this endpoint.

Student OTP Flow (Current — INSECURE)

- Frontend sends phone number to /api/v1/auth/send-otp
- Backend creates User row — no OTP is actually generated or sent
- Frontend sends any value to /api/v1/auth/verify-otp
- Backend returns user data regardless of OTP value

Provider OTP Flow (Current — Supabase)

- Provider login uses supabase.auth.signInWithOtp({ phone })
- Supabase generates and delivers the OTP (via configured SMS provider)
- Supabase verifies the OTP and creates a session
- Backend separately verifies otpVerified on ProviderProfile after a /verify-otp call

Recommended OTP Architecture

- Use Supabase Auth phone OTP for ALL users (students + providers)
- Configure Send SMS Hook to route OTP delivery through 2Factor.in or similar Indian SMS provider
- Rate limit: max 3 OTP requests per phone per 5 minutes
- Attempt limit: max 5 verification attempts per OTP
- OTP expiry: 5 minutes (Supabase default)
- CAPTCHA: Enable hCaptcha on Supabase Auth for abuse prevention

SECTION 10

SESSION / JWT / COOKIE ARCHITECTURE

Current Session State

[WARNING]

There are no server-side sessions, no JWTs verified by the backend, no secure cookies. User identity is stored in localStorage (userId, phone) and sent as custom headers.

What Currently Exists

- Supabase Auth client stores tokens in localStorage (sb-<ref>-auth-token)
- Student userId stored in localStorage — sent as x-user-id header
- Provider phone stored in localStorage — sent as x-provider-phone header
- No HttpOnly cookies are set by the backend
- No CSRF protection is needed (no cookies = no CSRF)
- No token rotation or refresh mechanism on the backend

Key Conceptual Clarifications

CONCEPT	DEFINITION	STAYVEO STATUS
JWT	Signed token containing user claims	Used by Supabase client only, not verified by backend
Session	Server-side record linking user to state	NOT IMPLEMENTED
Cookie	Browser-stored value sent with every request	NOT USED by backend
Redis Session	Session stored in Redis for scaling	NOT IMPLEMENTED
Access Token	Short-lived JWT for API authorization	Supabase issues these, backend ignores them
Refresh Token	Long-lived token to renew access tokens	Handled by Supabase client only

[RECOMMENDED]

RECOMMENDED: After implementing Supabase JWT verification on the backend, configure the frontend to send the Supabase access token as Authorization: Bearer <token>. This eliminates the need for custom x-user-id headers and provides cryptographic identity verification.

SECTION 11

AUTHORIZATION ARCHITECTURE

Authorization determines what an authenticated user is allowed to do. It is separate from authentication (which determines who the user is).

Current Authorization State

[WARNING]

Authorization is PARTIAL. Some provider routes enforce ownership checks, but there is no middleware-level authorization. Student routes have minimal or no authorization checks.

Provider Ownership (Tiffin — Implemented)

The `tiffin-provider.service.ts` contains a `resolveOwner()` function that:

- Resolves `ProviderProfile` from `x-provider-phone` header
- Verifies `otpVerified` flag on the profile
- Optionally matches `x-user-id` against `profile.userId`
- Resolves `TiffinKitchen` via `ownerId FK`
- All operations are scoped to the resolved kitchen

Student Routes (Minimal Authorization)

Most student routes accept `x-user-id` without verification. There is no check that the `userId` corresponds to an authenticated session.

SECTION 12

RBAC MODEL

Current Roles

The database defines two roles via the UserRole enum: STUDENT and PROVIDER. There is no ADMIN role.

ROLE	CAN ACCESS	ENFORCED?
STUDENT	PG listings, tiffin discovery, reservations, own profile	NO — any user ID is accepted
PROVIDER	Own kitchen/PG, own customers, own deliveries, own reports	PARTIAL — phone-based ownership
ADMIN	Kitchen verification, user management, system config	NOT IMPLEMENTED — role does not exist

Recommended RBAC Model

For StayVeo's current complexity, a simple Role + Resource Ownership model is more appropriate than full ABAC (Attribute-Based Access Control). Recommended roles:

- STUDENT — access own resources, public listings
- PROVIDER — access own kitchen/PG resources
- ADMIN — access all resources, verification, user management

SECTION 13 OWNERSHIP MODEL

Actual Database Ownership Chain

Based on the Prisma schema, the ownership hierarchy is:

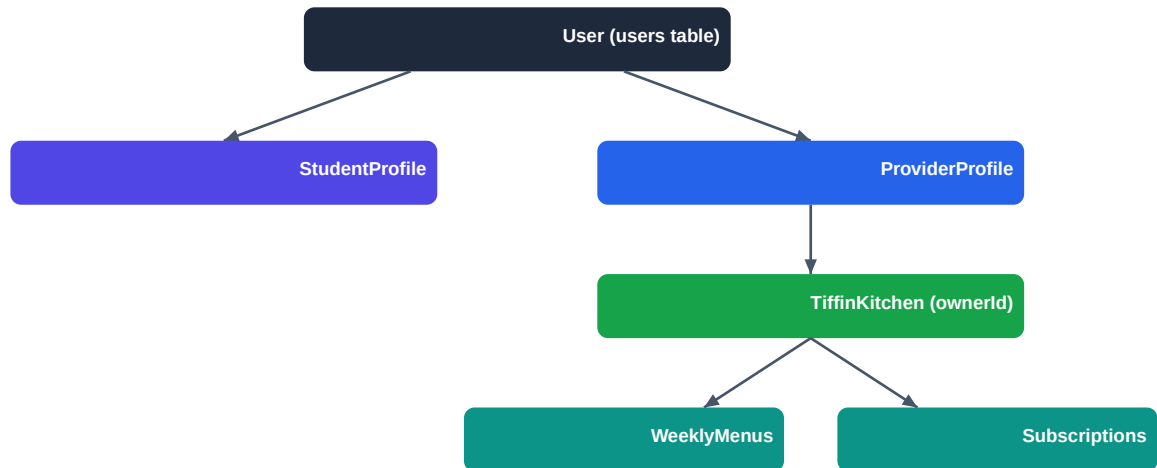


Figure 13.1 — Resource Ownership Chain

IDOR Prevention

Insecure Direct Object Reference (IDOR) occurs when a user can access another user's resources by changing an ID in the API request. The current tiffin provider routes prevent IDOR by resolving the kitchen from the authenticated provider's profile rather than accepting a kitchen ID from the request body. This is a correct pattern.

[CURRENT]

The `tiffin-provider.service.ts resolveOwner()` pattern is a good ownership enforcement example. However, it relies on trusting the `x-provider-phone` header. Once backend authentication is added, this pattern should resolve ownership from the verified JWT identity.

SECTION 14

API SECURITY

Current API Security Controls

CONTROL	STATUS	DETAILS
HTTPS/TLS	PARTIAL	Dev tunnels use HTTPS; no production TLS configured
Authentication	NOT IMPLEMENTED	No JWT/session verification middleware
Authorization	PARTIAL	Provider ownership in service layer only
Input Validation	PARTIAL	Zod on some routes; manual in tiffin-provider
Rate Limiting	NOT IMPLEMENTED	No rate limiting on any endpoint
Request Size	NOT CONFIGURED	Fastify defaults apply (~1MB)
CORS	CURRENT	Configured in app.ts; allows localhost + tunnels
CSRF	N/A	No cookies used, so no CSRF risk currently
Security Headers	NOT IMPLEMENTED	No helmet/security headers plugin
SQL Injection	LOW RISK	Prisma parameterizes queries; raw SQL in tiffin.repository.ts uses Prisma.sql tagged templates
Error Handling	CURRENT	Global handler masks Prisma/Zod errors
API Versioning	CURRENT	/api/v1 prefix on all routes
Idempotency	PARTIAL	TiffinPayment has idempotencyKey field
Logging	PARTIAL	Pino logger (stdout only, no persistent storage)

SECTION 15

REDIS / CACHING ARCHITECTURE

[NOTE]

STATUS: NOT IMPLEMENTED. No Redis, Memcached, or any caching layer exists in the current codebase.

Redis is an in-memory data store commonly used for caching, session storage, rate limiting, and queuing. StayVeo does not currently need Redis — it should be introduced when specific bottlenecks are identified.

When StayVeo Should Add Redis

- Rate Limiting — when auth endpoints need brute-force protection (Phase 9)
- Session Store — if server-side sessions are preferred over JWT (Optional)
- Cache — public tiffin listings, kitchen menus (when query latency becomes a bottleneck)
- Queue — replace in-memory notification queue with durable queue (when reliability matters)

What to Cache vs. Not Cache

DATA	CACHE?	REASON
Public tiffin listing results	YES	Read-heavy, changes infrequently (TTL: 5 min)
Today's menu for a kitchen	YES	Read-heavy, changes daily (TTL: 1 hour)
College list	YES	Static data (TTL: 24 hours)
User profile data	NO	Private, user-specific, changes frequently
Subscription details	NO	Sensitive, must be real-time accurate
Payment status	NO	Financial data, must be real-time
Delivery logs	NO	Frequently mutated, provider-specific

SECTION 16

DATABASE ARCHITECTURE

StayVeo uses PostgreSQL hosted on Supabase. The database schema is managed via Prisma ORM with 1312 lines of schema definition across 40+ models.

Database Connection Architecture

- DATABASE_URL: Supabase session-mode pooler (port 5432)
- DIRECT_URL: Same connection (used for Prisma migrations)
- Prisma Client: Singleton pattern with globalThis caching for hot-reload
- Connection logging: Error-level only

Major Entity Groups

GROUP	TABLES	RELATIONS
Identity	users, student_profiles	User !' StudentProfile (1:1)
Provider Core	provider_profiles, providers, provider_services	User !' ProviderProfile (1:1)
PG Module	pg_details, room_listings, pg_rooms	ProviderProfile !' RoomListing (1:N)
Tiffin Kitchen	tiffin_kitchens, kitchen_images, meal_timings	ProviderProfile !' Kitchen (1:1)
Tiffin Menu	weekly_menus, menu_histories	Kitchen !' Menu (1:N), unique day+meal
Tiffin Plans	subscription_plans	Kitchen !' Plans (1:N)
Tiffin Subscriptions	customer_subscriptions, subscription_days	Kitchen !' Subscription (1:N)
Tiffin Meals	meal_logs, meal_deliveries	Subscription !' MealLog (1:N)
Payments	tiffin_payments, payments, receipts	Subscription !' Payment (1:N)
Notifications	notifications, templates, preferences	User !' Notification (1:N)
Bookings	bookings, visit_requests	User !' Booking (1:N)
Discovery	tiffin_services, colleges, saved_listings	Legacy + current listings

Key Database Patterns

- UUID primary keys: All tables use gen_random_uuid() for IDs
- Soft deletes: TiffinKitchen has deletedAt; most tables use hard deletes
- Timestamps: created_at (auto), updated_at (Prisma @updatedAt)
- Unique constraints: User.phone_number, Kitchen.ownerId, Menu day+meal+kitchen
- Cascading deletes: Kitchen !' Menus, Plans, Subscriptions
- Indexed columns: Kitchen lat/lng, subscription status+endDate, meal logs

SECTION 17

DATABASE SECURITY / RLS

[WARNING]

STATUS: NOT VERIFIED. Row Level Security (RLS) status could not be confirmed without direct database access. The Prisma schema does not define RLS policies (Prisma does not manage RLS). The Supabase migrations directory exists but was not inspected for RLS SQL.

Current State

The backend connects to PostgreSQL using a connection string that likely uses the postgres role (service role), which bypasses RLS. All access control is therefore enforced at the application layer (Fastify services), not at the database layer.

RLS Design Considerations

[RECOMMENDED]

IMPORTANT: RLS policies should be designed AFTER authentication identity mapping is finalized. RLS policies depend on `auth.uid()` matching a user record, which requires the backend to connect with a per-user role or use Supabase's auth context — not the service role.

If the backend uses the service role (which bypasses RLS), then RLS only protects against direct Supabase client access from the frontend. Since the frontend uses the Supabase anon key for storage and real-time, RLS on `storage.objects` is critical.

SECTION 18

STORAGE SECURITY

Current Storage Implementation

Supabase Storage	[CURRENT]
File upload from frontend	[CURRENT]
Signed upload URLs for KYC	[CURRENT]
Storage RLS policies	[NOT VERIFIED]

Storage Buckets

- pg-images — PG listing photos (public bucket, uploaded from frontend)
- provider-kyc-documents — KYC documents (private bucket, signed upload URLs from backend)

Upload Flows

PG Images: Frontend !' Supabase Storage (direct upload with anon key)

KYC Documents: Frontend !' Backend (request signed URL) !' Backend !' Supabase (create signed URL) !' Frontend !' Supabase Storage (upload with signed URL)

[CURRENT]

The KYC upload flow correctly uses server-side signed URLs, preventing unauthorized uploads. The path includes the provider ID, ensuring document ownership. File type validation accepts only JPEG, PNG, and PDF.

[WARNING]

PG image uploads go directly from the frontend to Supabase Storage using the anon key. Without proper RLS policies on the pg-images bucket, any authenticated user could potentially upload to or overwrite files in other providers' directories.

SECTION 19

SECRETS MANAGEMENT

[CRITICAL]

Secrets are stored in plaintext .env files in the repository. The .env files are present in the working directory (and may be committed to git). This is acceptable for local development but MUST NOT be deployed to production.

Secret Classification

SECRET	EXPOSURE	RISK
DATABASE_URL (password)	Backend .env	Full database access if leaked
SUPABASE_SERVICE_ROLE_KEY	Backend .env	Bypasses all RLS, full admin access
VITE_SUPABASE_ANON_KEY	Frontend .env + bundle	Safe — designed for public use with RLS
VITE_MAPBOX_TOKEN	Frontend .env + bundle	Low — maps API with domain restrictions
CLOUDINARY_API_SECRET	Backend .env	Can manipulate/delete all Cloudinary assets

NOTE: Actual secret values are intentionally excluded from this document. The above table describes the types and risks, not the values.

Recommendations

- Move production secrets to environment variables set by the hosting platform
- Never commit .env files to git (verify .gitignore)
- Rotate DATABASE_URL password periodically
- Rotate SUPABASE_SERVICE_ROLE_KEY if it has been exposed
- Use separate Supabase projects for development and production

SECTION 20

LOGGING & MONITORING

[NOTE]

STATUS: PARTIAL. Fastify uses Pino logger with pino-pretty for development. No persistent log storage, no metrics collection, no alerting, no tracing.

Current Logging

- Pino logger configured in buildApp() — debug level in dev, info in production
- pino-pretty for colorized console output in development
- Unhandled errors logged via console.error in global error handler
- No structured log shipping to external service

Recommended Observability Stack

COMPONENT	TOOL	WHEN NEEDED
Logs	Pino !' Loki / CloudWatch / Datadog	Before production launch
Metrics	Prometheus + Grafana / Datadog	When traffic grows beyond dev testing
Tracing	OpenTelemetry !' Jaeger / Tempo	When debugging cross-service latency
Uptime	UptimeRobot / Better Stack	Immediately at production launch
Alerts	PagerDuty / Slack webhooks	When 24/7 reliability is required
Error Tracking	Sentry	Before production launch (frontend + backend)

SECTION 21 BACKUP & DISASTER RECOVERY

[NOTE]

STATUS: RELYING ON SUPABASE. Supabase provides automatic daily backups on paid plans and point-in-time recovery (PITR) on Pro+ plans. No custom backup strategy is configured.

Supabase Backup Capabilities

- Free plan: No automatic backups
- Pro plan (\$25/mo): Daily backups, 7-day retention
- Pro plan with PITR add-on: Point-in-time recovery

Recovery Objectives

METRIC	DEFINITION	TARGET
RPO	Maximum acceptable data loss	24 hours (daily backup) !' 1 hour (PITR)
RTO	Maximum acceptable downtime	4 hours (manual restore) !' 1 hour (automated)

SECTION 22

THREAT MODEL

This threat model is based on analysis of the actual StayVeo codebase and identifies realistic attack vectors relevant to this application.

THREAT	VECTOR	IMPACT	LIKELIHOOD	MITIGATION
Account Takeover	Send arbitrary x-user-id header	CRITICAL	HIGH	Add JWT verification middleware
Provider Impersonation	Send arbitrary x-provider-phone	CRITICAL	HIGH	Verify Supabase JWT, map to provider
OTP Brute Force	Dummy OTP accepts any value	HIGH	HIGH	Replace with Supabase Auth OTP
IDOR on Student	Change userId in saved/bookings	HIGH	HIGH	Verify userId from JWT, not header
Data Leakage	Error messages expose DB codes	MEDIUM	MEDIUM	Sanitize error responses (partially done)
SQL Injection	Malformed input in raw queries	HIGH	LOW	Prisma.sql templates (currently used correctly)
XSS	Inject script in menu items/names	MEDIUM	LOW	React auto-escapes; validate input length
DDoS	Flood API endpoints	HIGH	MEDIUM	Add rate limiting + CDN/WAF
Malicious Upload	Upload non-image to KYC bucket	MEDIUM	LOW	Backend validates content type (current)
Secret Leakage	.env committed to git	CRITICAL	MEDIUM	Verify .gitignore, rotate exposed keys
Session Theft	Steal localStorage userId	HIGH	MEDIUM	Replace with HttpOnly cookie or JWT
Privilege Escalation	Student accesses provider routes	HIGH	HIGH	Add role-based middleware

SECTION 23

SECURITY TESTING STRATEGY

[NOTE]

STATUS: NOT IMPLEMENTED. The repository contains no test files — no unit tests, integration tests, or security tests.

Recommended Testing Checklist

1. Authentication Tests: Verify JWT validation rejects invalid/expired tokens
2. Authorization Tests: Verify student cannot access provider routes and vice versa
3. IDOR Tests: Verify user A cannot access user B's resources by changing IDs
4. Ownership Tests: Verify provider A cannot modify provider B's kitchen
5. Input Validation: Test boundary values, special characters, oversized payloads
6. Rate Limiting: Verify OTP endpoint blocks after threshold
7. RLS Tests: Verify database policies enforce row-level access
8. Dependency Scanning: npm audit, Snyk, or GitHub Dependabot
9. Secret Scanning: Ensure no secrets in committed code (git-secrets, truffleHog)

SECTION 24

SCALING STRATEGY

Stage 1: Development / MVP (Current)

- Single Fastify server process
- Supabase Free/Pro plan
- Dev Tunnels for mobile testing
- No CDN, no WAF, no load balancer — NOT NEEDED YET

Stage 2: Early Production (0–1,000 users)

- Deploy to Railway / Render / Fly.io (single container)
- Add CDN for frontend (Vercel / Cloudflare Pages)
- Add Sentry for error tracking
- Add UptimeRobot for health monitoring
- Supabase Pro plan for daily backups

Stage 3: Growing (1,000–10,000 users)

- PM2 cluster mode (multi-process on single machine)
- Add Redis for rate limiting and caching
- Add Cloudflare WAF (free tier)
- Consider read replica for heavy query patterns

Stage 4: Scale (10,000+ users)

- Kubernetes / ECS with multiple API replicas behind ALB
- Managed Redis (Upstash / ElastiCache)
- Full observability stack (Datadog / Grafana Cloud)
- Database connection pooling tuning
- Consider dedicated PostgreSQL if Supabase limits are hit

[RECOMMENDED]

PRINCIPLE: Scale only when a measured bottleneck justifies the complexity and cost. Do not pre-optimize for traffic you do not have.

SECTION 25

COST STRATEGY

COMPONENT	DEV (FREE)	SMALL PROD	GROWING
Frontend Host	Vite dev	Vercel Free	Vercel Pro (\$20/mo)
Backend Host	localhost	Railway (\$5/mo)	Railway/Fly (\$20-50/mo)
Database	Supabase Free	Supabase Pro (\$25/mo)	Supabase Pro + PITR
CDN	None	Cloudflare Free	Cloudflare Free
WAF	None	Cloudflare Free	Cloudflare Pro (\$20/mo)
Redis	None	None	Upstash Free !' \$10/mo
Monitoring	None	UptimeRobot Free	Sentry + Better Stack
SMS/OTP	None (dummy)	2Factor.in (~ 10.15/SMS)	2Factor.in
Storage	Supabase Free	Supabase Pro (incl.)	Supabase Pro
TOTAL	\$0	~\$55/mo	~\$120-200/mo

SECTION 26

CURRENT ARCHITECTURE SNAPSHOT

This section captures the exact state of the StayVeo architecture as of the repository inspection date. All classifications are based on codebase analysis.

Current Security Posture

AREA	STATUS	DETAIL
Backend Auth Middleware	NOT IMPLEMENTED	No JWT/session checking on any route
OTP Verification	DUMMY (Student)	Accepts any OTP value for students
OTP Verification	SUPABASE (Provider)	Real Supabase OTP for providers
Provider Ownership	IMPLEMENTED	resolveOwner() in tiffin-provider.service.ts
Student Authorization	NOT IMPLEMENTED	x-user-id header trusted blindly
CORS	IMPLEMENTED	Configured with allowed origins
Input Validation	PARTIAL	Zod on auth; manual in tiffin service
Error Handling	IMPLEMENTED	Global handler with Prisma/Zod/generic
Rate Limiting	NOT IMPLEMENTED	No rate limiting anywhere
Security Headers	NOT IMPLEMENTED	No helmet or custom headers
RLS Policies	NOT VERIFIED	Cannot confirm without DB access
Storage Security	PARTIAL	KYC: signed URLs; PG images: anon upload
Logging	PARTIAL	Pino stdout only, no persistence
Tests	NOT IMPLEMENTED	No test files in repository

SECTION 27

TARGET ARCHITECTURE

The target architecture adds security, observability, and scalability layers while preserving the existing application structure.

Target Architecture Diagram

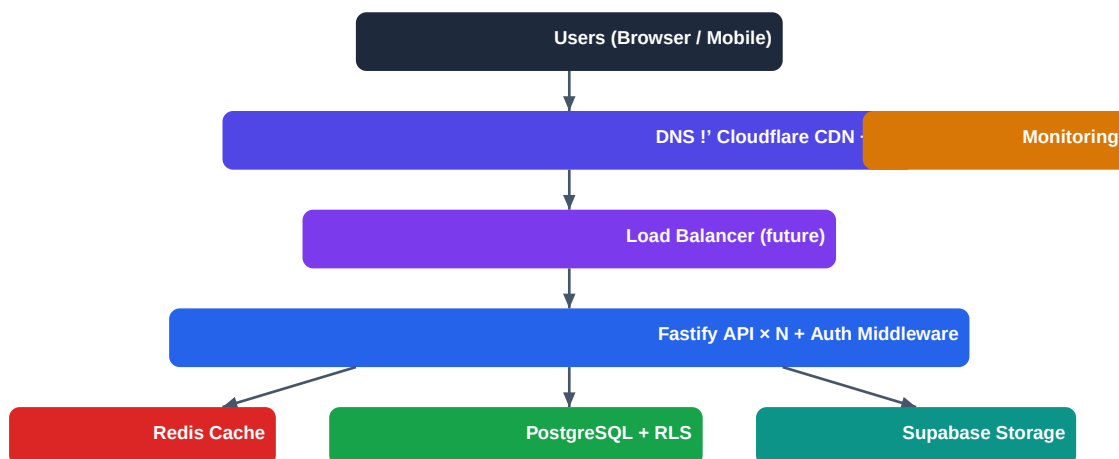


Figure 27.1 — Target Production Architecture

SECTION 28

MIGRATION PLAN

This phased roadmap prioritizes security-critical dependencies first. Each phase can be completed independently, with clear rollback strategies.

#	PHASE	KEY TASKS	PRIORITY
0	Architecture & Inventory	Document current state, classify all routes, map identity chain	IMMEDIATE
1	Identity / Authentication	Unify auth on Supabase Auth for all users, replace dummy OTP, add JWT middleware	CRITICAL
2	Backend Auth Middleware	Create Fastify auth plugin that verifies Supabase JWT on all /api/v1/* routes	CRITICAL
3	Authorization / RBAC	Add role-based route guards, separate student/provider/admin access	HIGH
4	Ownership Enforcement	Refactor resolveOwner() to use JWT identity, apply to all resource routes	HIGH
5	Database Constraints	Audit FK constraints, add missing indexes, enforce data integrity	MEDIUM
6	RLS Policies	Design and apply RLS for Supabase client access (storage, real-time)	MEDIUM
7	Storage Security	Add RLS to pg-images bucket, validate uploads server-side	MEDIUM
8	API Security Hardening	Add helmet, request size limits, security headers, CORS tightening	HIGH
9	Rate Limiting + Redis	Add Redis, rate limit auth/OTP endpoints, add brute-force protection	HIGH
10	Observability	Add Sentry, structured logging, uptime monitoring, basic alerts	MEDIUM
11	CDN / WAF	Deploy frontend to Vercel/CF Pages, proxy API through Cloudflare	LOW
12	Load Balancing	Add reverse proxy, PM2 cluster, then container orchestration	LOW
13	Backup / DR	Upgrade to Supabase Pro, enable PITR, test restore procedure	MEDIUM

SECTION 29

ARCHITECTURE DECISION RECORDS

ADR-001: Supabase as Backend Infrastructure

- Context: Need managed PostgreSQL, auth, and storage without ops overhead
- Decision: Use Supabase for database, auth, and file storage
- Reason: Reduces operational complexity; free tier supports MVP; includes auth and storage
- Alternatives: Self-hosted PostgreSQL, Firebase, PlanetScale
- Consequence: Vendor lock-in to Supabase; must work within Supabase connection limits

ADR-002: Phone OTP as Primary Authentication

- Context: Indian student users prefer phone-based auth over email/password
- Decision: Use phone OTP for all user authentication
- Reason: Higher conversion rate; matches Indian market expectations (Paytm, GPay pattern)
- Alternatives: Email/password, Google OAuth, Magic links
- Consequence: SMS cost per login; requires reliable SMS delivery in India

ADR-003: Separate Authentication from Authorization

- Context: System has multiple user types with different permissions
- Decision: Auth middleware verifies identity; separate checks enforce permissions
- Reason: Clean separation of concerns; easier to test and audit
- Alternatives: Combined auth+authz middleware
- Consequence: Requires two middleware layers; but simpler individual logic

ADR-004: Server-Side Ownership Enforcement

- Context: Providers must not access other providers' kitchens
- Decision: Resolve owner from verified identity, not from request parameters
- Reason: Prevents IDOR attacks; client cannot manipulate ownership
- Alternatives: Trust client-provided kitchenId (insecure)
- Consequence: Extra database lookup per request (acceptable cost)

ADR-005: RLS After Identity Mapping

- Context: RLS policies depend on knowing which database user maps to which auth identity
- Decision: Implement RLS only after auth middleware maps JWT to user rows
- Reason: RLS without proper auth context is ineffective or creates false security
- Alternatives: Implement RLS immediately without backend auth
- Consequence: RLS is delayed; but when implemented, it works correctly

ADR-006: Redis Optional Initially

- Context: Early-stage product with low traffic
- Decision: Do not add Redis until a specific bottleneck is identified
- Reason: Avoids premature complexity and cost; in-memory solutions work for early traffic
- Alternatives: Add Redis from day 1
- Consequence: Rate limiting relies on in-process counters initially (acceptable for dev)

SECTION 30

FAILURE SCENARIOS

FAILURE	DETECTION	USER IMPACT	RECOVERY
API Server Crash	Health check fails	All API calls fail, app unusable	Auto-restart (PM2/container), no data loss
Database Down	Prisma connection error	All reads/writes fail	Supabase auto-recovery; check status page
Redis Down	Connection timeout	Cache miss !' DB fallback; rate limits reset	Degrade gracefully; restart Redis
SMS Provider Down	OTP delivery fails	New users cannot sign up	Fallback SMS provider; notify support
Supabase Storage Down	Upload/download 5xx	Images not loading, KYC upload fails	Retry with backoff; Supabase status page
CDN Down	Static assets 5xx	Frontend not loading	DNS failover to origin; rare with Cloudflare
10x Traffic Spike	CPU/memory spike	Slow responses, timeouts	Scale out API servers; enable rate limiting
Auth Service Down	JWT verification fails	Cannot authenticate new requests	Cache valid tokens briefly; fallback mode

SECTION 31

SECURITY GAP REGISTER

ID	GAP	SEVERITY	CURRENT STATE	FIX
SG-01	No backend auth middleware	CRITICAL	x-user-id header trusted	Add JWT verification middleware
SG-02	Dummy OTP (accepts any)	CRITICAL	auth.service.ts dummy mode	Use Supabase Auth OTP
SG-03	No rate limiting	HIGH	All endpoints unlimited	Add rate limiter (Redis-backed)
SG-04	No security headers	HIGH	No helmet/HSTS/CSP	Add @fastify/helmet plugin
SG-05	Secrets in .env files	HIGH	Plaintext in working dir	Use platform env vars
SG-06	No automated tests	HIGH	Zero test coverage	Add auth/authz/IDOR tests
SG-07	No ADMIN role	MEDIUM	Admin ops require DB access	Add ADMIN role and panel
SG-08	PG image upload auth	MEDIUM	Anon key upload to Storage	Add RLS or signed URLs
SG-09	No log persistence	MEDIUM	Pino to stdout only	Ship logs to external service
SG-10	No backup verification	MEDIUM	Rely on Supabase free tier	Upgrade + test restore

SECTION 32

ARCHITECTURE MATURITY ASSESSMENT

Scoring: 0 = Not implemented | 1 = Basic | 2 = Partial | 3 = Good | 4 = Production Ready | 5 = Mature

AREA	SCORE	JUSTIFICATION
Authentication	1/5	Supabase Auth exists for providers but not enforced server-side; dummy OTP for students
Authorization	2/5	Provider ownership checks exist; no role-based middleware; student routes unprotected
API Security	2/5	CORS configured; Zod validation on some routes; no rate limits or security headers
Database Security	2/5	Prisma prevents SQL injection; FK constraints and unique indexes; RLS unverified
RLS	0/5	Not verified; likely not enforced due to service-role connection
Storage Security	2/5	KYC uses signed URLs (good); PG images use anon upload (needs RLS)
Secrets Management	1/5	Secrets in .env files; no rotation; no vault
Observability	1/5	Pino logger to stdout; no metrics, tracing, or alerting
Scalability	1/5	Single server; no clustering, caching, or load balancing
Backup / DR	1/5	Relying on Supabase free tier; no tested restore procedure
Testing	0/5	No test files in repository
Error Handling	3/5	Comprehensive global handler for Prisma/Zod/generic errors

[NOTE]

OVERALL MATURITY: Pre-Production (Early Development). The application architecture is sound for an MVP, but significant security hardening is required before handling real user data and payments.

SECTION 33

REQUEST LIFECYCLE EXAMPLES

Example 1: Student Opens Tiffin Listing

1. Browser navigates to /tiffin !' React SPA renders TiffinListing component
2. Component calls API: GET /api/v1/tiffin (via frontend client.js)
3. Request hits Fastify server on port 3000
4. tiffin.routes.ts !' tiffinController.list
5. tiffin.service.ts !' tiffinRepository.findFiltered()
6. Repository executes raw SQL CTE via prisma.\$queryRaw (with Haversine distance)
7. PostgreSQL returns rows via Supabase session pooler
8. Response: { success: true, data: { items: [...], pagination: {...} } }
9. React renders listing cards

Example 2: Provider Publishes Menu

1. Provider clicks "Publish Menu" on /provider/tiffin/menu
2. Frontend calls PUT /api/v1/tiffin/provider/menu with merged menu data
3. x-provider-phone and x-user-id headers are sent
4. tiffin-provider.controller.ts !' saveMenu()
5. resolveOwner() resolves ProviderProfile from phone, verifies otpVerified
6. Kitchen resolved via ownerId FK (ownership enforced)
7. Existing menus pre-fetched in single findMany query
8. Transaction: create history records + upsert menu items atomically
9. Response returns updated menu; frontend shows success toast

SECTION 34

SECURITY PRINCIPLES

% **Zero Trust**

Never trust any request by default. Verify every identity, every time, regardless of network location.

% **Least Privilege**

Grant only the minimum permissions needed. Students should not access provider routes; providers should not access other providers' data.

% **Defense in Depth**

Layer multiple security controls (WAF !' auth !' authorization !' validation !' RLS). If one fails, others still protect.

% **Secure by Default**

New routes should require authentication by default. An opt-out (public) annotation should be explicit.

% **Fail Closed**

If authentication fails, deny access. Never fall through to an open/public state.

% **Never Trust Client Input**

All input from the browser is untrusted. Validate, sanitize, and escape everything server-side.

% **Never Trust Client Identity**

The client cannot prove who it is via custom headers alone. Use cryptographic tokens (JWT) verified server-side.

% **Secrets Never in Frontend**

Service-role keys, database passwords, and API secrets must never appear in client-side JavaScript bundles.

% **Minimize Attack Surface**

Remove unnecessary routes, disable debug endpoints in production, hide server version headers.

SECTION 35

GLOSSARY

TERM	DEFINITION
DNS	Domain Name System — translates domain names (stayveo.com) to IP addresses
CDN	Content Delivery Network — caches static files at edge locations worldwide for faster delivery
WAF	Web Application Firewall — inspects and filters malicious HTTP traffic before it reaches the server
DDoS	Distributed Denial of Service — attack that overwhelms a server with massive traffic
Load Balancer	Distributes incoming requests across multiple server instances
Reverse Proxy	Server that forwards requests to backend servers; hides origin details
API Gateway	Entry point that handles routing, rate limiting, and authentication for APIs
Authentication	Verifying WHO a user is (identity proof)
Authorization	Verifying WHAT a user is allowed to do (permission check)
RBAC	Role-Based Access Control — permissions assigned to roles (STUDENT, PROVIDER)
JWT	JSON Web Token — digitally signed token containing user identity claims
RLS	Row Level Security — PostgreSQL feature that restricts which rows a user can access
ORM	Object-Relational Mapping — library that maps database tables to code objects (Prisma)
Prisma	Type-safe ORM for Node.js/TypeScript that generates database queries from a schema
TLS	Transport Layer Security — encrypts data in transit (HTTPS)
CORS	Cross-Origin Resource Sharing — controls which domains can call the API
CSRF	Cross-Site Request Forgery — attack that tricks a browser into making unwanted requests
XSS	Cross-Site Scripting — attack that injects malicious scripts into web pages
IDOR	Insecure Direct Object Reference — accessing resources by manipulating IDs in requests
Rate Limiting	Restricting how many requests a client can make in a time period
SAST	Static Application Security Testing — analyzing source code for vulnerabilities
DAST	Dynamic Application Security Testing — testing a running application for vulnerabilities
RPO	Recovery Point Objective — maximum acceptable data loss in a disaster
RTO	Recovery Time Objective — maximum acceptable downtime after a failure

SECTION 36

FINAL RECOMMENDATIONS

What Should We Implement First?

The following roadmap is ordered by security criticality and dependency chain. Each item builds on the previous one.

Ø=Ý4 P0 Replace dummy OTP with Supabase Auth phone OTP for all users

Anyone can impersonate any user currently

Ø=Ý4 P0 Add JWT verification middleware to all API routes

No authentication exists on the backend

Ø=ßà P1 Add role-based route guards (STUDENT vs PROVIDER)

Students can call provider endpoints and vice versa

Ø=ßà P1 Refactor identity resolution to use JWT (not headers)

Headers can be spoofed by any HTTP client

Ø=ßà P1 Add rate limiting on auth/OTP endpoints

Prevent brute-force and abuse

Ø=ßà P2 Add security headers (helmet) and tighten CORS

Basic web security hygiene

Ø=ßà P2 Add Sentry error tracking (frontend + backend)

Production visibility into errors

Ø=ßà P2 Write auth/authorization integration tests

Prevent security regression

Ø=ßà P3 Add RLS to Supabase Storage buckets

Protect uploaded files from unauthorized access

Ø=ßà P3 Deploy to production hosting with proper domain

Eliminate dev tunnel dependency

Ø=Ý5 P4 Add Redis for caching and rate limiting

Performance and security at scale

Ø=Ý5 P4 Set up CDN + WAF (Cloudflare)

Edge protection and performance

[CRITICAL]

DO NOT deploy to production with real user data until at minimum P0 items (JWT authentication middleware + real OTP) are completed. The current codebase allows any HTTP client to impersonate any user.

— End of Document —